

Deep learning with convolutional neural networks

Hervé Le Borgne
Yannick Le Cacheux

2021-2022

Content

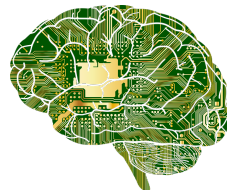
- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Convolutional networks

- Convolutional networks (conv nets) deal with data organised in a grid:
 - ▶ sampled signals (time series...)
 - ▶ images
- They are currently the most widely used "deep networks" and those which have led to some of the most spectacular applications.
- In particular, they were at the origin of the craze of the "third connectionist wave" (*deep learning*) following:
 - ▶ theoretical advances to make large architectures converge [Hinton *et al.*, 2006]
 - ▶ exponential growth in available data (*smart phones*, web 2.0...)
 - ▶ computing power (GPGPU) adapted to their architecture



Convolutional networks

- Significantly improved state of the art in areas where the base "material" (data) is close to the raw signal:
 - ▶ supervised classification
 - ▶ fields "close to the signal"
 - ★ speech recognition (time-frequency representation)
 - ★ computer vision (pixels)
- Less spectacular for "semantic processing" (NLP)
 - ▶ But still, improvements in machine translation
- They have a "biological inspiration", with an analogy with the brain of vertebrates (not only humans...)



What color are her eyes?
What is the mustache made of?



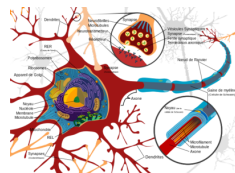
How many slices of pizza are there?
Is this a vegetarian pizza?



Is this person expecting company?
What is just under the tree?



Does it appear to be rainy?
Does this person have 20/20 vision?



Content

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Definition

A Convolutional Neural Network (CNN) is a neural network in which at least one layers uses a convolution instead of the usual matrix product ("fully connected" multi-layer perceptron (MLP))

- there exist "purely" convolutional networks
- in practice, convolutions are often used before fully connected layers

Convolution (mathematical definition)

If functions f and ω are integrable (more specifically Lebesgues-integrable) over $\mathbb{R}$, their convolution is:

$$(f * \omega)(x) = \int_{-\infty}^{+\infty} f(t)\omega(x - t) dt \quad (1)$$

For discrete functions:

$$(f * \omega)(n) = \sum_{m=-\infty}^{m=+\infty} f(m)\omega(n - m) \quad (2)$$

2D version:

$$(f * \omega)(x, y) = \int_{-\infty}^{+\infty} f(t, u)\omega(x - t, y - u) dt du \quad (3)$$

Vocabulary

$$(f * \omega)(x) = \int f(t)\omega(x - t) dt \quad (4)$$

- f is the input signal (image)
- ω is a *kernel*
- $f * \omega$ is *feature map*
- In practice:
 - ▶ the input is usually multidimensional (e.g an image has three channels R,G,B). We thus deal with tensors.

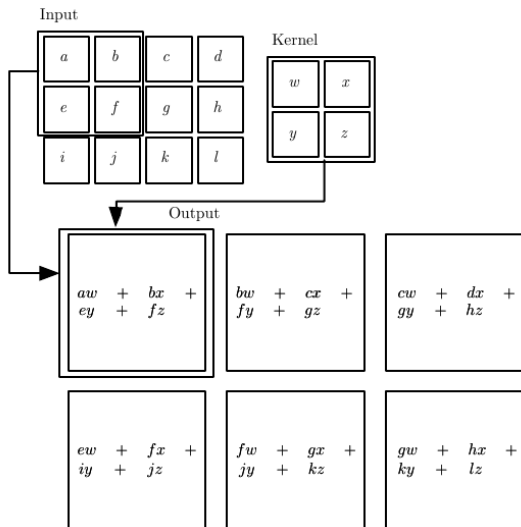
Convolution in practice in CNNs

Spoiler: convolutional networks generally do not make use of convolutions!
They actually use **cross-correlation**:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) \quad (5)$$

- By convention, we still use the term "convolution"

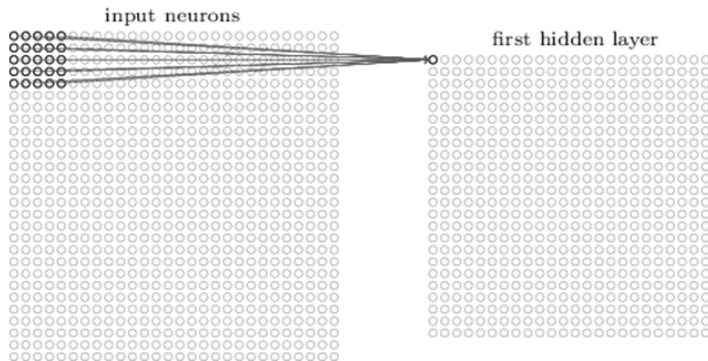
How convolutions work



source [Goodfellow et al., 2016]

How convolutions work

From "farther away":



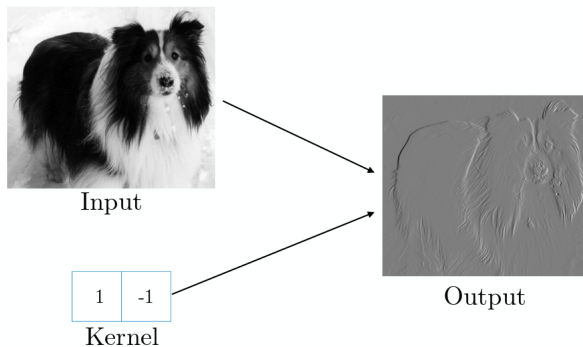
Visualization of 5 x 5 filter convolving around an input volume and producing an activation map

source

adeshpande3.github.io

Example

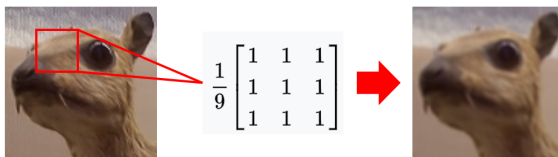
We can use the following kernel to detect (vertical) edges:



Horizontal edges could be detected in a similar way (how?)

Example

This kernel will apply a blur filter:



- These are just basic examples to help you understand what's going on
- Remember: in CNNs, the parameters of the kernels are *learned*

Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - **Interesting properties**
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Interesting properties

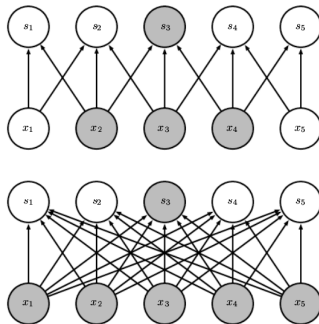
The success of CNNs comes from:

- "sparse" interactions
→ computational efficiency
- parameter sharing
→ memory gains
→ statistical robustness
- the equivariant representation property
→ translational invariance

Sparse interactions

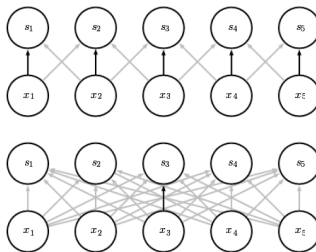
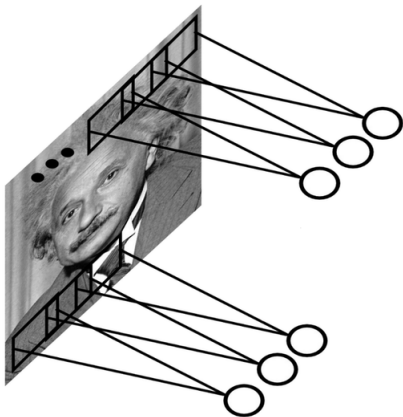
Vocabulary: *sparse interactions*, *sparse connectivity* or *sparse weights*

- The layers from a MLP are *fully connected*
 - ▶ Each of the m inputs interacts with each of the n outputs
 - ▶ $m \times n$ parameters need to be estimated (complexity $O(m \times n)$)
- The *conv* layers are limited to k connections with $k \ll m, n$
 - ▶ fewer parameters to estimate – memory efficiency
 - ▶ more robust statistical estimation (with the same data)



Sharing of parameters

- MLP: each parameter is evaluated independently
- ConvNets: the parameters of the kernels are "shared" and used at multiple locations of the image
 - ▶ the *local* statistical properties can be reused elsewhere
- → this improves the robustness of the estimation



source [Ranzato, CVPR 2012] [Goodfellow et al, 2016]

Sharing the parameters and translation equivariance

Equivariance

Function f is equivariant with respect to g if:

$$f(g(x)) = g(f(x))$$

Property

Convolution is equivariant w.r.t. translation.

Convolution (or actually: cross correlation) of I by K is (in 1D to simplify notations):

$$(I * K)(x) = \sum_u I(u)K(u - x) \quad (6)$$

If we translate the result by t (pixels):

$$(I * K)(x - t) = \sum_u I(u)K(u - x + t) \quad (7)$$

Sharing the parameters and translation equivariance

If we translate the result by t (pixels):

$$(I * K)(x - t) = \sum_u I(u)K(u - x + t) \quad (8)$$

Let L_t be the translation by t (pixels). $[L_t I](x) = I(x - t)$

$$([L_t I] * K)(x) = \sum_u I(u - t)K(u - x) \quad (9)$$

$$= \sum_{u'} I(u')K(u' + t - x) \quad (10)$$

$$= (I * K)(x - t) \quad (11)$$

$$= [L_t [I * K]](x) \quad (12)$$

→ Translating then applying a convolution is equivalent to applying a convolution then translating

Property

Convolution is equivariant with respect to translation

Translating then applying a convolution is equivalent to applying a convolution then translating

→ Detecting a localized pattern independently from time or location



NOT a property

Convolution is NOT equivariant to rotation, zoom, etc.

→ We need to apply specific processing to deal with these transformations

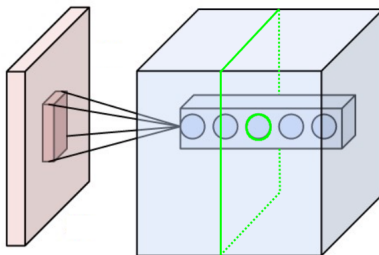
- Process input data to make it invariant
 - ▶ ex: find the "main axis" of the image, and rotate the image so that the main axis is vertical (→ rotational invariance)
- Feed the transformed data to the CNN
 - ▶ This is related to data augmentation (later)



Plan

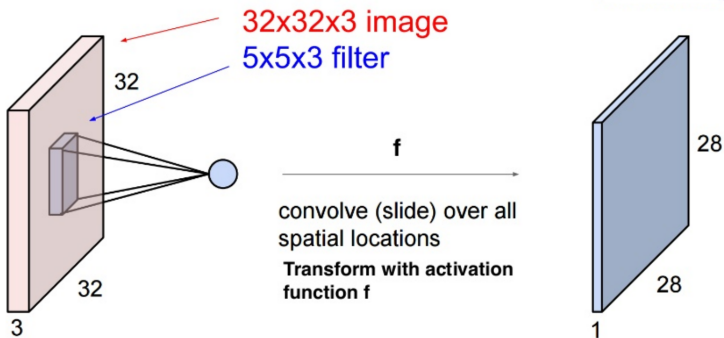
- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

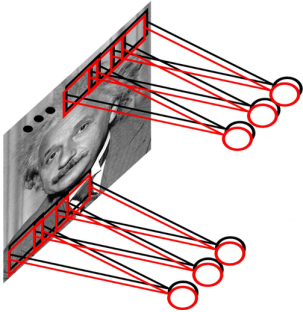
- A convolution with *one* kernel extracts *one* type of feature
- for neural *networks*, we generally want *several* types of features
→ we use several kernels in parallel
- one *layer* consists of neurons organised in 3D:
 - ▶ the width and height are the same
 - ▶ the feature maps are stacked in 3D



source: Karpathy

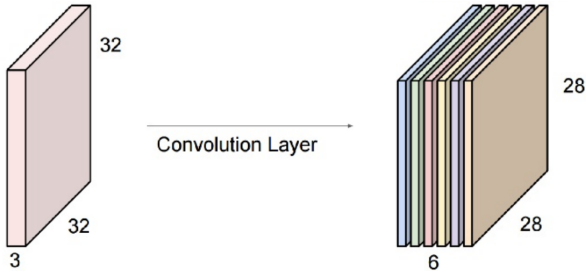
- The result of a convolution over an image is a *feature map*
- The spatial extent of the kernel is the *receptive field*
 - ▶ it has a fixed height/width (e.g. 5)
 - ▶ it takes the whole input depth (e.g 3) into account





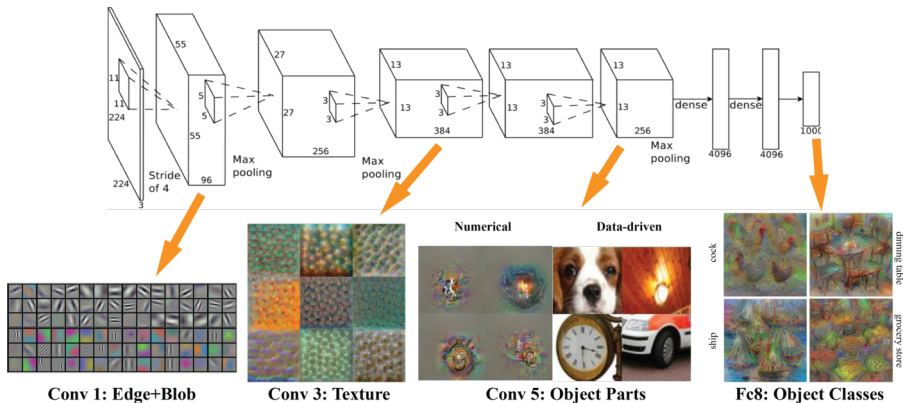
- one convolution layer "learns" several kernels in parallel
 - ▶ since they are initialized differently, they learn different things
(in order to decrease the training loss)

- this leads to a "stack" of features maps



Feature maps

- On images, features can represent increasingly complex or abstract information



D. Wei et al., Understanding Intra-Class Knowledge inside CNN, Arxiv:1507.02379, 2015

Convolutions in detail

- The output of a convolution layer is a 3D volume
- It is the input of the following layer
- In practice, the first layer of data is often a volume
 - ▶ an image is a volume of size $H \times W \times 3 = H \times W \times (R, G, B)$
 - ▶ the depth of this volume is the number of (*channels*)
- If we have
 - ▶ an input (3D) $V_{i,j,k} = V_{channel_in,row,col}$
 - ▶ kernels (4D) $K_{a,b,c,d} = K_{channel_out,channel_in,offset_row,offset_col}$
(we consider $a = channel_out$ kernels in parallel)
 - ▶ an output (3D) $Z_{i,j,k} = Z_{channel_out,row,col}$

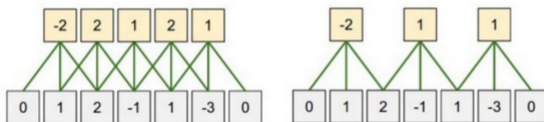
The output is equal to:

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n} \quad (13)$$

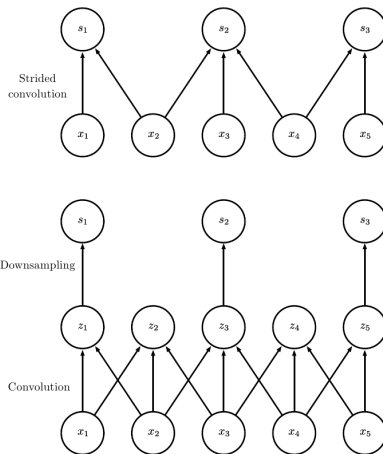
"Incomplete" convolution (*stride*)

- We can "avoid" some spatial positions
 - ▶ decreases the computing cost
 - ▶ estimation of features is less precise
- This is equivalent to under-sampling the full output
- We define a (*stride*) s

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,(j-1) \times s + m, (k-1) \times s + n} K_{i,l,m,n} \quad (14)$$



- The stride s can be different depending on the direction: s_{row}, s_{col}

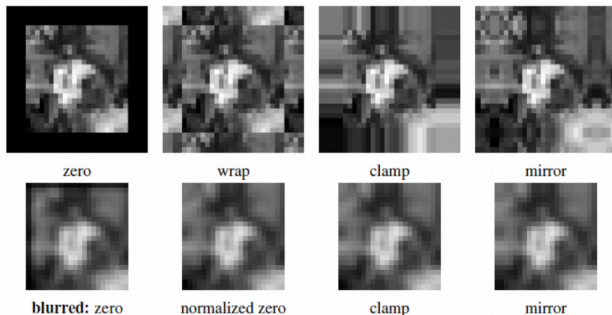


- a convolution with a stride $s = 2$ yields the same output as the full convolution ($s = 1$) followed by under sampling
- but it is more computationally efficient

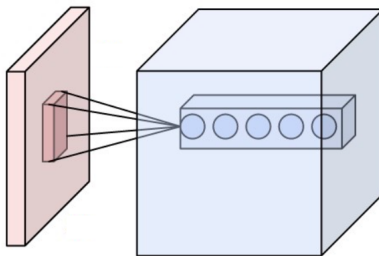
Dealing with image boundaries

Several possibilities

- Don't do anything (this will reduce the size of the image).
- Add 0 or any constant value (*zero padding*).
- Use symmetry: $f(-x, y) = f(x, y)$.
- No solution is better than all the others.



Exercise



Practice: given a kernel, identify

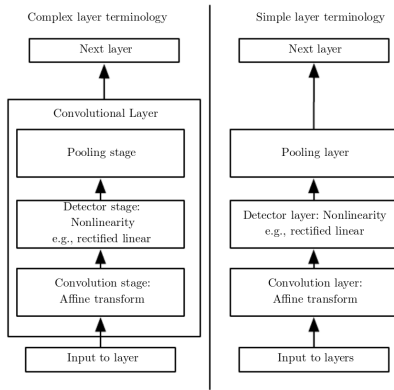
- The number of learned parameters
- The size of the feature maps
- The receptive field of a kernel

Typical layers in a CNN:

- convolution: a linear transformation producing feature maps
- non linearities (activation functions)
- aggregations (*pooling*) in a (spatial) neighborhood

The term "layer" can actually denote **two types** of layers:

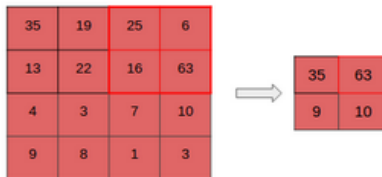
- complex: layers with multiples stages
- simple: each processing step.
Some layers don't have parameters



Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

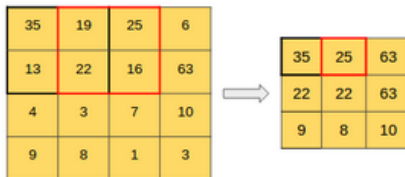
Aggregation (pooling): example



Example of max pooling:

- with *stride* [2 2]
- with *stride* [1 1]

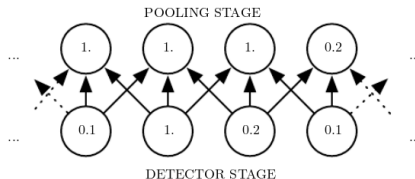
(iii)



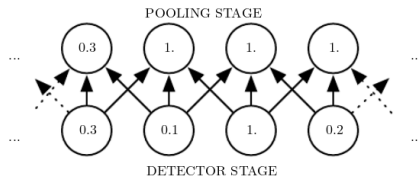
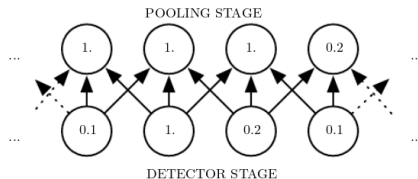
Aggregation (pooling)

Aggregation "summarizes" a neighborhood

- After a layer, each value is replaced by a "summary" of its neighborhood
 - ▶ Ex. of a "neighborhood": small rectangular window (2×2 ; 3×3 ...)
- Several types of *pooling*:
 - ▶ *max pooling*: maximum value in the neighborhood
 - ▶ *average pooling*: mean value in the neighborhood
 - ▶ *L_2 norm*: $(\sum_{\text{neigh.}} x_i^2)^{1/2}$
 - ▶ *weighed avg pooling*: average weighted by the distance to the center

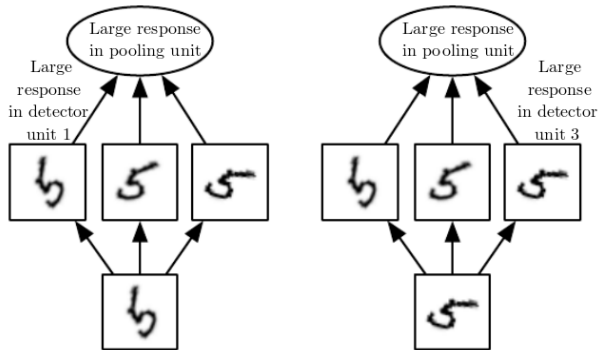


- enables an invariance w.r.t. small translations



The aggregation of outputs over several convolutional layers enable to learn complex invariant patterns.

Example with rotation [Goodfellow *et al.*, 2013]



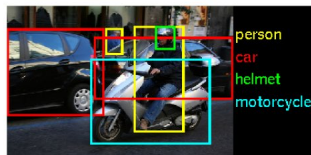
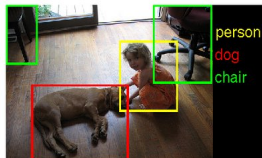
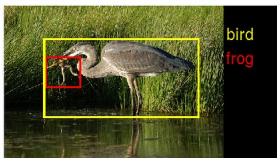
→ identical response to both inputs (rotations of one another)

Plan

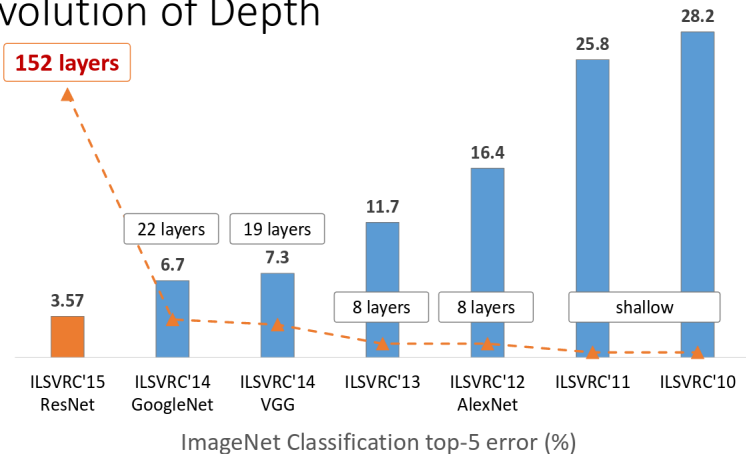
- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Challenge ILSVRC

- ImageNet Large Scale Visual Recognition Challenge (2010 – 2017)
- ImageNet:
 - ▶ 10M images from +10k categories
 - ▶ hierarchical information (→ wordnet)
- *Localization & classification*: box around the 5 main objects
- *Detection*: from 0 to objects; false positives are penalized



Revolution of Depth



ImageNet Classification top-5 error (%)

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015.

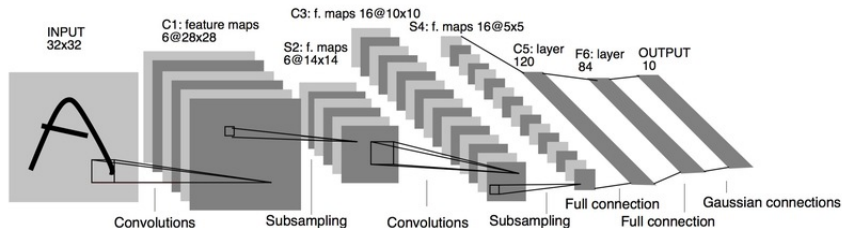


How did we get here?

LeNet5

[LeCun et al., Proc. IEEE, 1998]

- 10 years of research (1988 – 1998) applied to bank check recognition (handwritten writing)



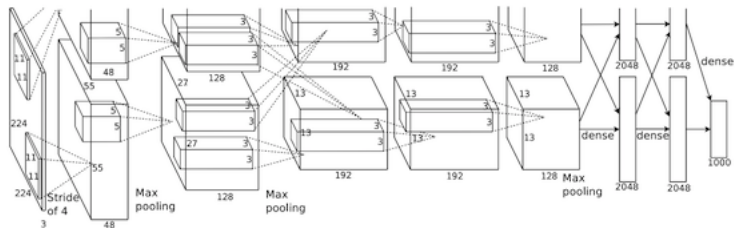
Several of the deep learning basic ideas:

- Extraction of feature by conv layer, classification with MLP
- Layer conv+pooling+non-linearity (sigmoid or tanh)
- Learning with back-propagation

AlexNet

[Krizhevsky et al., NIPS 2012]

- LeNet-5, but bigger, implemented on GPU
 - ▶ 60M parameters including $\sim 55M$ for the fully connected layers
- Rectified Linear Units (ReLU) non linearities
- Dropout to prevent overfitting
- Huge improvement at ILSVRC 2012 $\rightarrow$ revival of deep learning!
- CaffeNet implementation is available [Jia et al, Arxiv 2014]



VGG

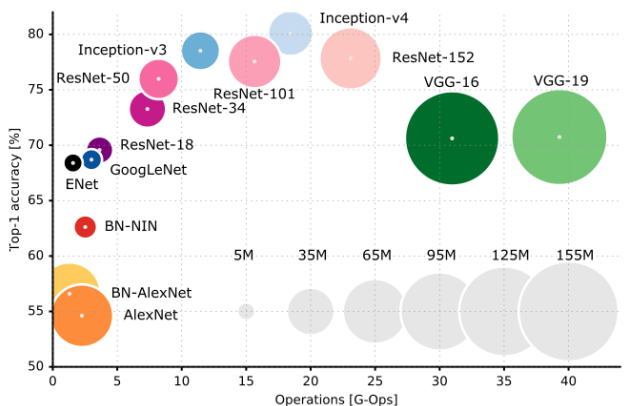
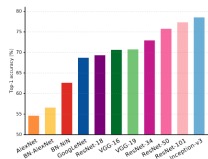
[Simonyan & Zisserman, Arxiv 2014, ICLR 2015]

- *very deep*: we now have 16 to 19 layers
- Multiple small filters instead of few big ones
 - ▶ More non linearity $\rightarrow$ more discrimination
 - ▶ decreases the number of parameters for a given receptive field

	3×3	7×7
configuration	3×3	
	3×3	
receptive field	7×7	7×7
# param (mapping $C \rightarrow C$ channels)	$3(3^2 C^2) = 27 C^2$	$7^2 = 49 C^2$

- Configurations from 11 to 19 layers
 - ▶ training is difficult: initialized with 11 layers, then more layers are added
- Use of 1×1 convolutions

Overview of architectures



- x axis: number of operations in the forward pass
- y axis: top-1 classif score on ImageNet challenge
- size of circle $\sim$ number of parameters

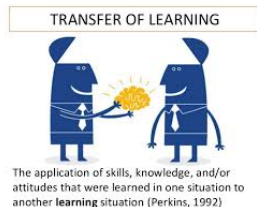
[Canziani et al. ArXiv 2016]

Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Practical issues

- Since CNN have millions of parameters, *a lot* of data is necessary to train them
 - ▶ Typically: ImageNet = 1.2M images (1k classes)
- Problem : the target problems (the interesting ones) do not always provide many labeled examples (**labeling is expensive**)
- Two main solutions:
 - ▶ Data augmentation
 - ▶ Transfer learning
- Related:
 - ▶ Fine tuning



Data augmentation

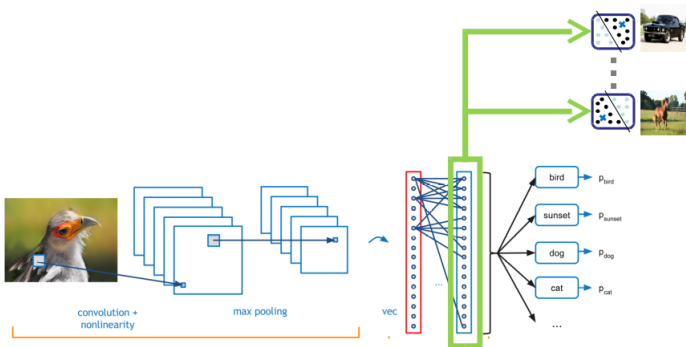
- Idea = increase the size of the training set by transforming data (images), replacing by synonyms (text), adding noise (sound)...
- Ex: many transformations do not modify the class of an object
 - ▶ labeling cost is 0 (same label)
 - ▶ collecting cost is 0 (available image + geometric transformation)



- Can also lead to a (relative) invariance to some transformations

Transfer learning

- It is sometimes impossible to train a CNN "from scratch" (i.e with random initialization) because we don't have enough data (even with data augmentation)
 - Ex: 10k images, transformed $\rightarrow$ 200k images $\ll$ 1.2M images
- We can initialize with a ***pre-trained*** network



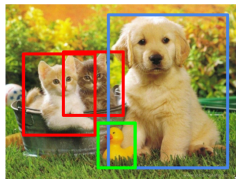
Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Detection

- Objective = find a bounding box around objects of interest
- How do we define the problem?

credit: Li Fei-Fei



Regression: varying size output



DOG, (x, y, w, h)
CAT, (x, y, w, h)
CAT, (x, y, w, h)
DUCK (x, y, w, h)

= 16 numbers



DOG, (x, y, w, h)
CAT, (x, y, w, h)

= 8 numbers



CAT, (x, y, w, h)
CAT, (x, y, w, h)
....
CAT (x, y, w, h)

= many numbers

Classification



CAT? NO

DOG? NO



CAT? YES!

DOG? NO



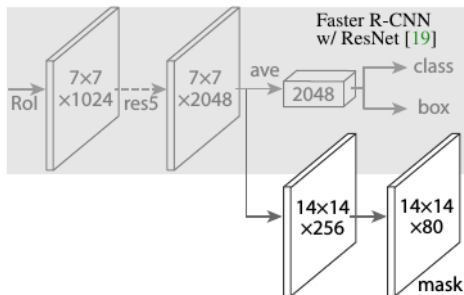
CAT? NO

DOG? NO

Semantic segmentation

[He et al., Mask R-CNN, ICCV 2017]

- Classification problem for each pixel in an image



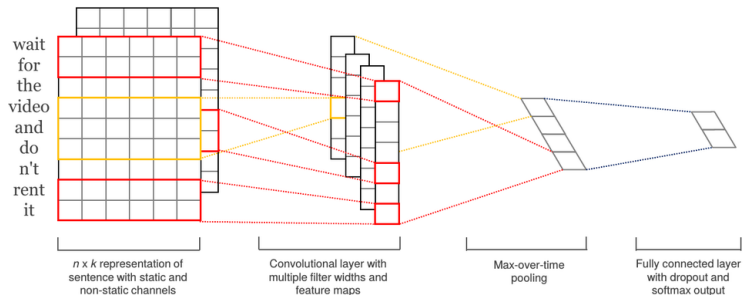
Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

CNNs for textual representation

[Kim, EMNLP 2014] [Zeng *et al*, COLING 2014]

- An N-gram (cf. cours 2) is an ordered set of N words
- It is a pattern identifiable by a convolutional kernel
 - ▶ size of the filter (1D) = number of words taken into account
- Each word can be represented by a dense vector (*embedding*)



Plan

- 1 Introduction
- 2 Convolution
 - Definition
 - Interesting properties
 - Convolutional layers
- 3 Aggregation (pooling)
- 4 Standard CNN architectures for classification
- 5 Transfer learning and data augmentation
- 6 Other convolutional architectures
 - Architectures for object detection
 - CNNs for textual representation
- 7 Conclusion

Conclusion

- Definition of convolutions
- Properties of convolutional layers
 - ▶ sparse interactions → efficient computing
 - ▶ parameters sharing → statistical robustness + memory
 - ▶ representation equivariance → translational invariance
- Pooling as a summary of the neighborhood
- Architectures for classification: AlexNet, VGG...
- Other architectures pour la detection, segmentation...
- In practice:
 - ▶ Data augmentation
 - ▶ Transfer learning

References

- Canziani A., Paszke A., Culurciello A. (2016) An Analysis of Deep Neural Network Models for Practical Applications, ArXiv 1605.07678
- Ciresan D.C., Ueli Meier, Luca Maria Gambardella, Juergen Schmidhuber (2010) « Deep Big Simple Neural Nets Excel on Handwritten Digit Recognition », arXiv:1003.0358
- He K., Zhang X., Ren S. and Sun J. (2015) Deep Residual Learning for Image Recognition, arXiv:1512.03385
- Hinton, G. E., Osindero, S., and Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. Neural Computation, 18.
- Hinton, G. (2007). To recognize shapes, first learn to generate images. Computational Neuroscience: Theoretical Insights into Brain Function., Elsevier.
- Krizhevsky A., Ilya Sutskever, Geoffrey E. Hinton (2012) ImageNet Classification with Deep Convolutional Neural Networks, Advances in Neural Information Processing Systems 25 (NIPS 2012)
- Jia Y., E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, T. Darrell, (2014) Caffe: Convolutional architecture for fast feature embedding, arXiv:1408.5093.
- Le Cun Y., L. Bottou, Y. Bengio, and P. Haffner (1998) Gradient-based learning applied to document recognition. Proceedings of the IEEE, november 1998.
- Liu W., Anguelov D., Erhan D., Szegedy C., Scott Reed, Cheng-Yang Fu, and Alexander C. Berg (2016) SSD: Single Shot MultiBox Detector, ECCV

References

- Conneau, A., Lample, G., Ranzato, M., Denoyer, L., & Jégou, H. (2018). Word Translation Without Parallel Data. ICLR
- Kim Y.(2014) Convolutional Neural Networks for Sentence Classification. EMNLP. 2014.
- Lin M., Qiang Chen, Shuicheng Yan (2014) Network In Network, ICLR 2014
- Ranzato, M., Huang, F., Boureau, Y. & LeCun, Y.(2007). Unsupervised Learning of Invariant Feature Hierarchies with Applications to Object Recognition. CVPR'07
- Ranzato, M., Poultney, C., Chopra, S. & LeCun, Y. (2006). Efficient Learning of Sparse Representations with an Energy-Based Model. NIPS 2006.
- Redmon J., Divvala S., Girshick R., Farhadi A. (2016) You Only Look Once: Unified, Real-Time Object Detection, CVPR, Dallas, USA
- Salakhutdinov, R. & Hinton, G. (2007). Learning a Nonlinear Embedding by Preserving Class Neighborhood Structure. AISTATS, 11
- Sanjeev Arora, Aditya Bhaskara, Rong Ge, and Tengyu Ma. Provable bounds for learning some deep representations. CoRR, abs/1310.6343, 2013

References

- Simard, P.Y., Steinkraus, D. & Platt, J.C. (2003). Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis. ICDAR 2003, pp 958 – 962.
- Simonyan K. & Zisserman A. (2014), Very Deep Convolutional Networks for Large-Scale Image Recognition arXiv technical report, 2014
- Sermanet P., Eigen D., Zhang X., Mathieu M., Fergus R., Le Cun Y. (2013) OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks. arXiv:1312.6229
- Steinkraus D., Ian Buck et Patrice Simard, (2005) « Using GPUs for Machine Learning Algorithms » ICDAR 2005, pp. 1115–1119
- Szegedy C., W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich (2014) Going Deeper with Convolutions, arXiv:1409.4842v1
- Szegedy C., Ioffe S., Vanhoucke V., Alemi A.(2016) Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning, arXiv:1602.07261
- M. Wulfmeier, A. Bewley, and I. Posner (2017) Addressing Appearance Change in Outdoor Robotics with Adversarial Domain Adaptation, in IEEE/RSJ International Conference on Intelligent Robots and Systems
- Zeng D. *et al* (2014) Relation Classification via Convolutional Deep Neural Network. COLING. 2014.